

ESP32C6-Based Email System Using SMTP Protocol with Attachment Support

Aim: The aim of this project is to design and implement an ESP32C6-based email system using the SMTP protocol that enables sending emails with plain text and file attachments through the Arduino IDE.

Components required:

- 1) ESP32C6 Dev Module
- 2) USB to Type C Cable
- 3) Arduino IDE

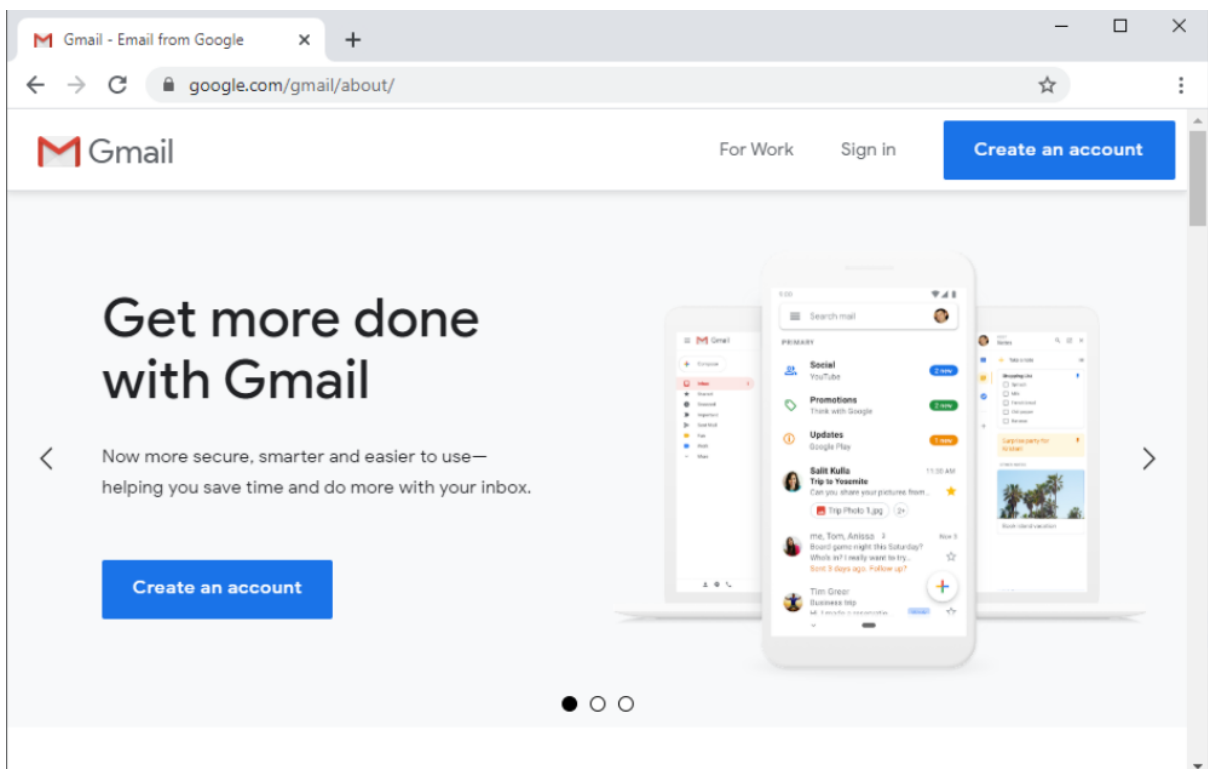
Introduction to SMTP Servers: SMTP means Simple Mail Transfer Protocol, and it is an internet standard for email transmission. To send emails using an ESP32C6, you need to connect it to an SMTP Server.

Connections in detail

- To send emails with the [ESP32](#), we'll use the [ESP-Mail-Client library](#). This library allows the ESP32C6 to send and receive emails with or without attachments via SMTP servers.
- In this manual, we'll use SMTP to send an email with and without attachments. As an example, we'll send an image (.png) and a text (.txt) file. The files sent via email can be saved in the ESP32C6 Filesystem (LittleFS).
- **Installing the ESP-Mail-Client Library:** you need to install the ESP-Mail_Client library.
- Go to **Sketch > Include Library > Manage Libraries** and search for **ESP Mail Client**. Install the *ESP Mail Client library by Mobizt*.



- **Sender Mail Instructions:** We recommend creating a new email account to send the emails to your main personal email address. Do not use your main personal email to send emails via ESP32C6. If something goes wrong in your code or if by mistake you make too many requests, you can be banned or have your account temporarily disabled.
- We'll use a newly created Gmail.com account to send the emails, but you can use any other email provider. The receiver email can be your personal email without any problem.
- **Create a Sender Email Account:** Create a new email account for sending emails with the ESP32. If you want to use a Gmail account, [go to this link](#) to create a new one.

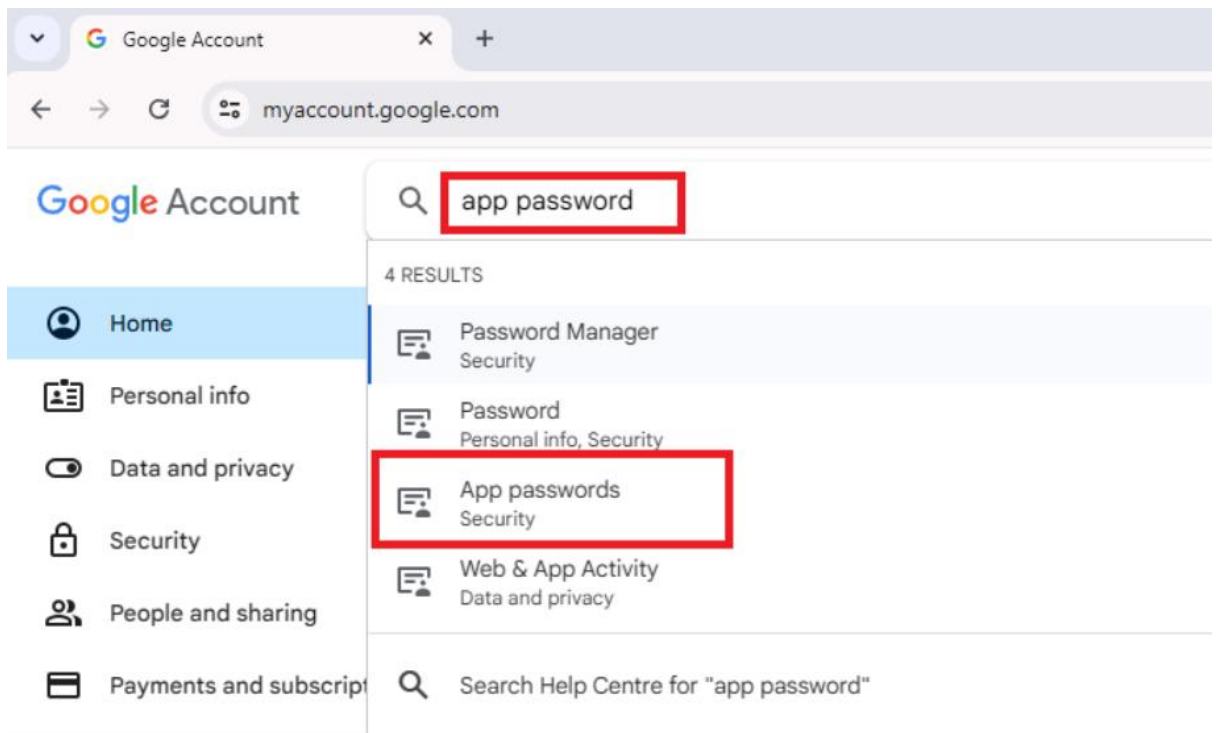


- **Create an App Password:** You need to create an app password so that the ESP32 is able to send emails using your Gmail account. An App Password is a 16-digit passcode that gives a less secure app or device permission to access your Google Account. [Learn more about sign-in with app passwords here.](#)
- An app password can only be used with accounts that have **2-step verification turned on.**
- Open your [Google Account](#).
- In the search panel, Select **Security**.
- Under “Signing in to Google,” select **2-Step Verification > Get started.**

- Follow the on-screen steps.

After enabling 2-step verification, you can create an app password.

- Open your [Google Account](#).
- In the search panel, search for **App Passwords**.
- Open the **App Passwords** menu.



- Give a name to the app password, for example **ESP**. Then, click on **Create**. It will pop-up a window with a password that you'll use with the ESP32 to send emails. Save that password (even though it says you won't need to remember it) because you'll need it later.

← App passwords

App passwords help you sign in to your Google Account on older apps and services that don't support modern security standards.

App passwords are less secure than using up-to-date apps and services that use modern security standards. Before you create an app password, you should check to see if your app needs this in order to sign in.

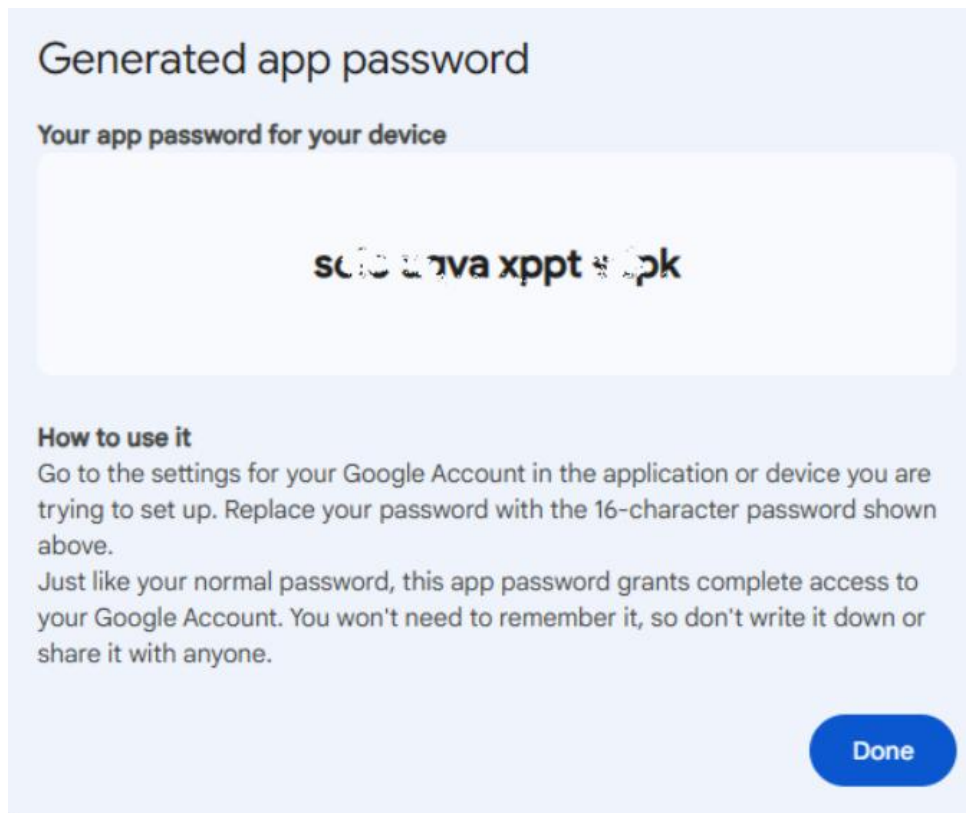
[Learn more](#)

You don't have any app passwords.

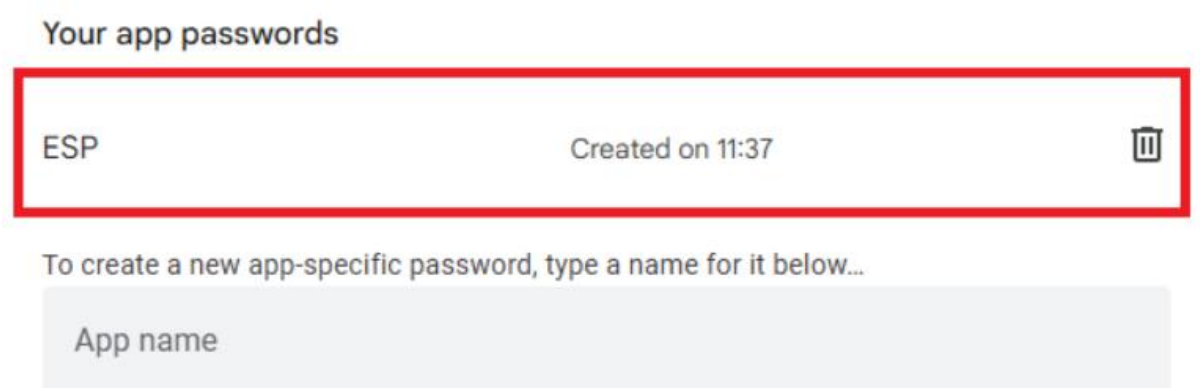
To create a new app-specific password, type a name for it below...

App name
ESP

- It will pop-up a window with a password that you'll use with the ESP32 to send emails. Save that password (even though it says you won't need to remember it) because you'll need it later.



- Now, you should have an app password that you'll use on the code to send the emails.



- **Gmail SMTP Server Settings you configure in your IDE**
 - SMTP Server: **smtp.gmail.com**
 - SMTP username: Complete Gmail address
 - SMTP password: Your Gmail password

- SMTP port (TLS): **587**
- SMTP port (SSL): **465**
- SMTP TLS/SSL required: **yes**

Full code:

```
#include <Arduino.h>
#include <WiFi.h>           // ESP32-C6 WiFi
#include <ESP_Mail_Client.h> // ESP Mail Client

/***** Wi-Fi Credentials *****/
#define WIFI_SSID   "Enter_Your_Name"
#define WIFI_PASSWORD "Enter_your_Password"

/***** SMTP Server Settings *****/
#define SMTP_HOST    "smtp.gmail.com"
#define SMTP_PORT     465

/***** Email Credentials *****/
#define AUTHOR_EMAIL  "author@gmail.com" //This is your sender email aka the
dummy email you made and configured just now
#define AUTHOR_PASSWORD "Your_16_digits_password" //Your dummy email's app
password which has 16 digits

/***** Recipient *****/
#define RECIPIENT_EMAIL "recipient@gmail.com" //this the reciever's email

/* SMTP session object */
SMTPSession smtp;

/* Callback declaration */
void smtpCallback(SMTP_Status status);

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\nESP32-C6 Email Sender");

  /***** Wi-Fi Connection *****/
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to Wi-Fi");

  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(300);
  }
}
```

```

Serial.println("\nWi-Fi connected");
Serial.print("IP address: ");
Serial.println(WiFi.localIP());

/***** Mail Client Settings *****/
MailClient.networkReconnect(true);
smtp.debug(1);          // 0 = off, 1 = basic debug
smtp.callback(smtpCallback);

/***** Session Configuration *****/
Session_Config config;

config.server.host_name = SMTP_HOST;
config.server.port      = SMTP_PORT;

config.login.email      = AUTHOR_EMAIL;
config.login.password   = AUTHOR_PASSWORD;
config.login.user_domain = "";

// NTP time (required for SSL)
config.time.ntp_server = F("pool.ntp.org,time.nist.gov");
config.time.gmt_offset = 3;    // Adjust to your timezone
config.time.day_light_offset = 0;

/***** Email Message *****/
SMTP_Message message;

message.sender.name = F("ESP32-C6");
message.sender.email = AUTHOR_EMAIL;
message.subject     = F("ESP32-C6 Test Email");
message.addRecipient(F("Recipient"), RECIPIENT_EMAIL);

// Plain text message
String textMsg = "Hello World!\nSent from ESP32-C6";
message.text.content = textMsg.c_str();
message.text.charSet = "us-ascii";
message.text.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

message.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_low;
message.response.notify =
    esp_mail_smtp_notify_success |
    esp_mail_smtp_notify_failure |
    esp_mail_smtp_notify_delay;

/***** Connect to SMTP Server *****/
if (!smtp.connect(&config)) {
    ESP_MAIL_PRINTF(

```

```

"SMTP Connection failed. Status: %d, Error: %d, Reason: %s\n",
smtp.statusCode(),
smtp.errorCode(),
smtp.errorReason().c_str()
);
return;
}

if (smtp.isAuthenticated()) {
    Serial.println("SMTP authenticated successfully");
} else {
    Serial.println("SMTP connected (no authentication)");
}

/***** Send Email *****/
if (!MailClient.sendMail(&smtp, &message)) {
    ESP_MAIL_PRINTF(
        "Send failed. Status: %d, Error: %d, Reason: %s\n",
        smtp.statusCode(),
        smtp.errorCode(),
        smtp.errorReason().c_str()
    );
}
}

void loop() {
    // Nothing needed here
}

/***** Callback *****/
void smtpCallback(SMTP_Status status) {
    Serial.println(status.info());

    if (status.success()) {
        Serial.println("-----");
        ESP_MAIL_PRINTF("Sent success: %d\n", status.completedCount());
        ESP_MAIL_PRINTF("Sent failed : %d\n", status.failedCount());
        Serial.println("-----");

        for (size_t i = 0; i < smtp.sendingResult.size(); i++) {
            SMTP_Result result = smtp.sendingResult.getItem(i);

            ESP_MAIL_PRINTF("Message #%%d\n", i + 1);
            ESP_MAIL_PRINTF("Status : %s\n", result.completed ? "Success" : "Failed");
            ESP_MAIL_PRINTF("Date/Time: %s\n",
                MailClient.Time.getDateTimeString(
                    result.timestamp,

```



```

"%B %d, %Y %H:%M:%S"
).c_str()
);
ESP_MAIL_PRINTF("Recipient: %s\n", result.recipients.c_str());
ESP_MAIL_PRINTF("Subject : %s\n", result.subject.c_str());
}

smtp.sendingResult.clear(); // Prevent memory growth
}
}

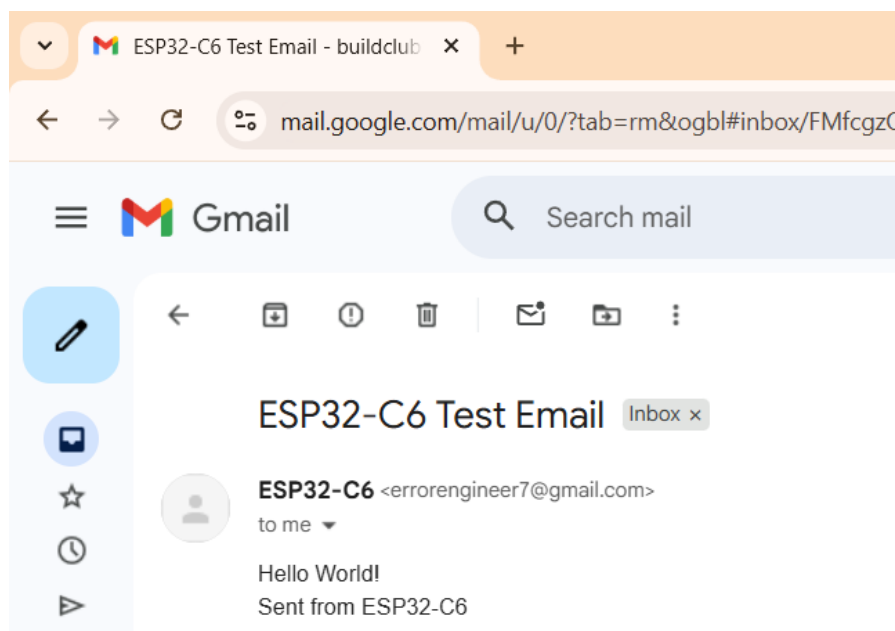
```

Result:

- Upload the code to your ESP32C6. After uploading, open the Serial Monitor at a baud rate of 115200. Press the ESP32C6 Reset button.
- If everything went as expected, you should get a similar message in the Serial Monitor.



- Check your email account. You should have received an email from your ESP32 board.



Email With Attachment

Steps in Detail:

- In this section, we'll show you how to send attachments in your emails sent by the ESP32. We'll show you how to send `.txt` files or pictures
- For this section, the files to be sent should be saved on the ESP32 filesystem (LittleFS).

Introducing LittleFS

- LittleFS is a lightweight filesystem created for microcontrollers that lets you access the flash memory as you do in a standard file system on your computer, but it's simpler and more limited. You can read, write, close, and delete files. Using LittleFS with the ESP32 boards is useful.

Install ESP32 LittleFS Uploader (Upload Files to the Filesystem)

- Now you'll learn how to upload files to the ESP32 LittleFS Filesystem using Arduino IDE
- To upload files to the ESP32 on Arduino IDE, we'll use this LittleFS Uploader plugin that is compatible with Arduino 2.2.1 or higher and can be used with the ESP32.
- Go to the [releases page](#) and click the `.vsix` file to download.

Release 1.6.3 - Fix for hangs on very large ESP32 uploads

Latest

What's Changed

- Use event driven STDERR/STDOUT dumping for tools by [@earlephilhower](#) in [#127](#)

Full Changelog: [1.6.2...1.6.3](#)

Contributors

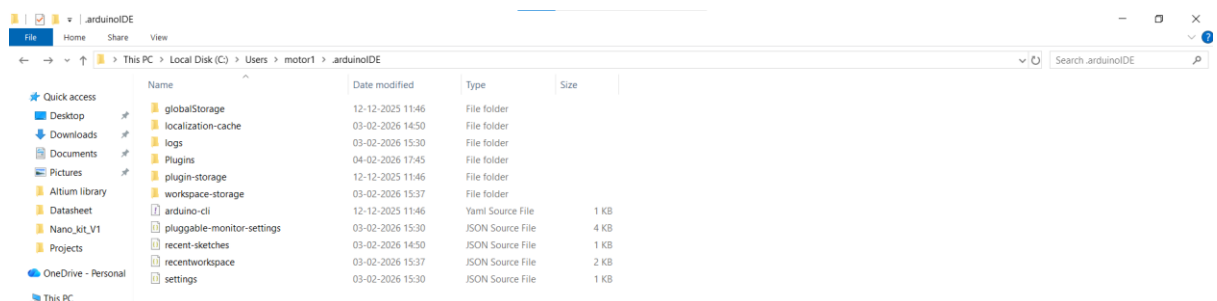


earlephilhower

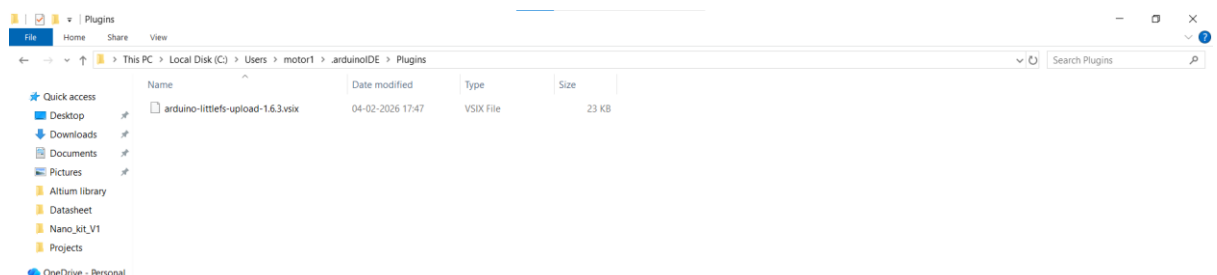
Assets 3

arduino-littlefs-upload-1.6.3.vsix	sha256:22c02d4f02cc09465...		22.9 KB	3 weeks ago
Source code (zip)				3 weeks ago
Source code (tar.gz)				3 weeks ago

- On your computer, go to the following path: C:\Users\<username>\.arduinoIDE\.
Create a new folder called plugins if you haven't already.

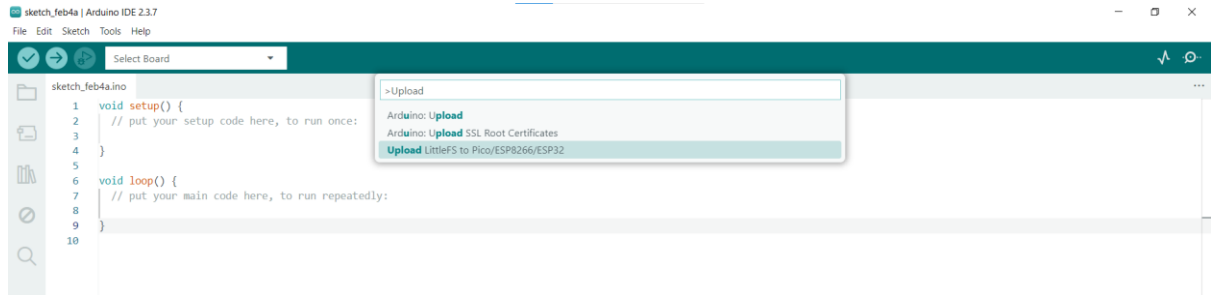


- Move the .vsix file you downloaded previously to the plugins folder (remove any other previous versions of the same plugin if that's the case).



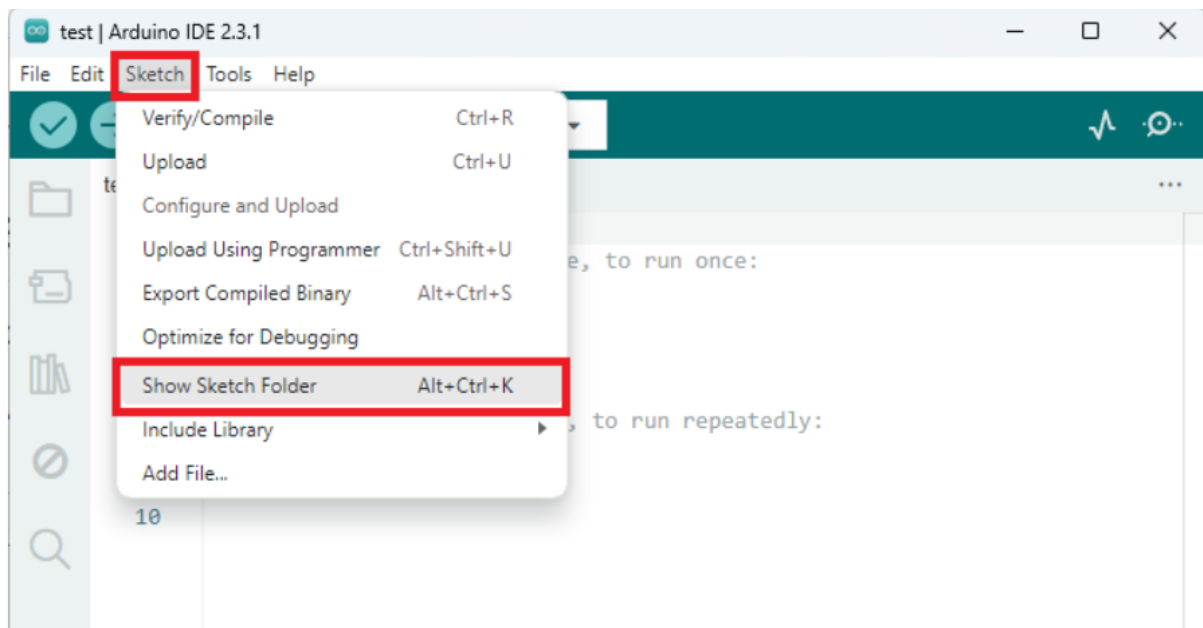
- Restart or open the Arduino IDE. To check if the plugin was successfully installed, press **[Ctrl] + [Shift] + [P]** to open the command palette. An instruction

called '**Upload Little FS to Pico/ESP266/ESP32**' should be there (just scroll down or search for the name of the instruction).



Uploading Files to ESP32 using the Filesystem Uploader in Arduino IDE

- To upload files to the ESP32 LittleFS filesystem, follow the next instructions.
- Create an Arduino sketch and save it. For demonstration purposes, you can save an empty sketch.
- Then, open the sketch folder. You can go to **Sketch > Show Sketch Folder**. The folder where your sketch is saved should open.



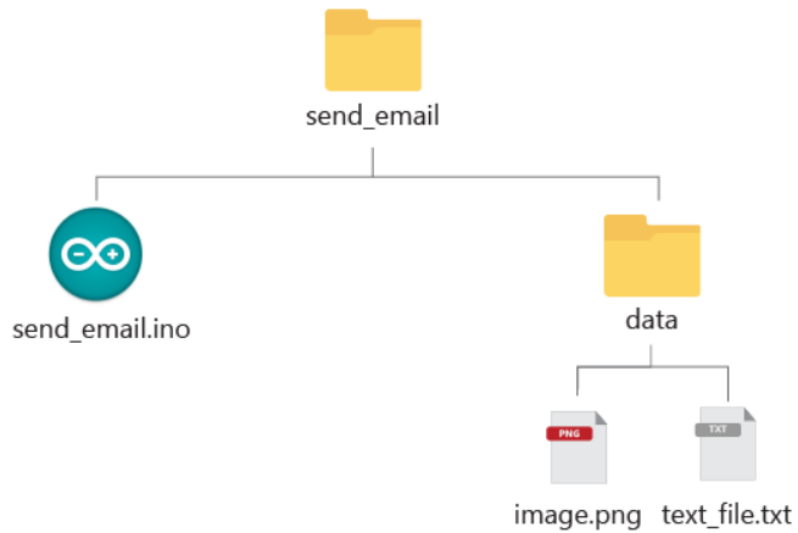
- Inside that folder, create a new folder called **data**.



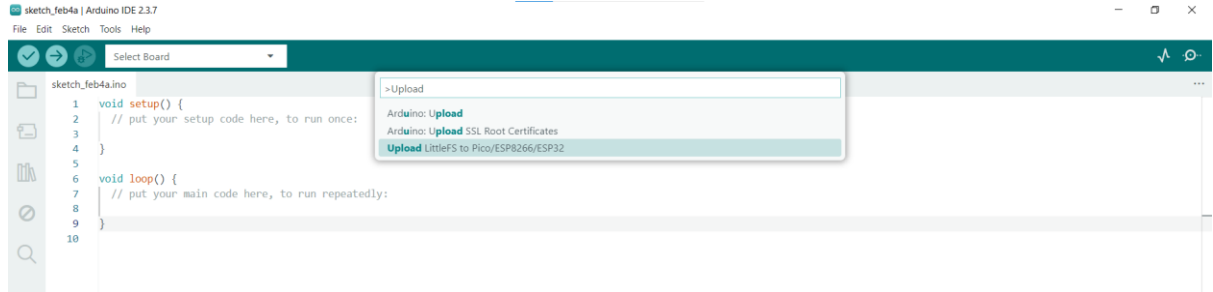
- Inside the folder Move a .jpg file and .txt file to your **data** folder.

Note: with the default code, *your files must be named image.png and text_file.txt*. Alternatively, you can modify the code to import files with a different name.

- Your folder structure should look as follows



- After moving the files to the **data** folder, you need to upload them to your board. In the Arduino IDE, press **[Ctrl] + [Shift] + [P]** to open the command palette. Search for the **Upload LittleFS to Pico/ESP8266/ESP32** command and click on it.



- You should get a success message on the debugging window. If the files were successfully uploaded, move on to the next section.

Full Code:

```
#include <Arduino.h>
#include <WiFi.h>
#include <ESP_Mail_Client.h>
#include <LittleFS.h>

/* WiFi credentials */
#define WIFI_SSID "YOUR_WIFI_NAME"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"

/* SMTP server settings */
#define SMTP_HOST "smtp.gmail.com"
#define SMTP_PORT 465 // SSL

/* Email credentials (Gmail APP PASSWORD only) */
#define AUTHOR_EMAIL "yourgmail@gmail.com"
#define AUTHOR_PASSWORD "your_16_digit_app_password"

/* Recipient */
#define RECIPIENT_EMAIL "receiver@gmail.com"

/* SMTP session */
SMTPSession smtp;

/* Callback */
void smtpCallback(SMTP_Status status);

void setup() {
```

```

Serial.begin(115200);
delay(1000);

/* WiFi connect */
Serial.print("Connecting to WiFi");
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
while (WiFi.status() != WL_CONNECTED) {
  Serial.print(".");
  delay(300);
}
Serial.println("\nWiFi connected");
Serial.println(WiFi.localIP());

/* LittleFS init (IMPORTANT for ESP32-C6) */
if (!LittleFS.begin(true)) {
  Serial.println("LittleFS Mount Failed");
  return;
}

/* Enable debug */
smtp.debug(1);
smtp.callback(smtpCallback);

/* Session config */
Session_Config config;
config.server.host_name = SMTP_HOST;
config.server.port = SMTP_PORT;
config.login.email = AUTHOR_EMAIL;
config.login.password = AUTHOR_PASSWORD;

/* Time (required for SSL cert validation) */
config.time.ntp_server = F("pool.ntp.org,time.nist.gov");
config.time.gmt_offset = 5.5 * 3600; // India
config.time.day_light_offset = 0;

/* Message */
SMTP_Message message;
message.sender.name = "ESP32-C6 Mail";
message.sender.email = AUTHOR_EMAIL;
message.subject = "ESP32-C6 Email Test";
message.addRecipient("Receiver", RECIPIENT_EMAIL);

message.html.content = "ESP32-C6 sending email with attachments using LittleFS ";
message.html.charSet = "utf-8";
message.html.transfer_encoding = Content_Transfer_Encoding::enc_qp;

/* Attachment: image */

```

```

SMTP_Attachment att;
att.descr.filename = "image.png";
att.descr.mime = "image/png";
att.file.path = "/image.png";
att.file.storage_type = esp_mail_file_storage_type_flash;
att.descr.transfer_encoding = Content_Transfer_Encoding::enc_base64;
message.addAttachment(att);

/* Attachment: text file */
message.resetAttachItem(att);
att.descr.filename = "text_file.txt";
att.descr.mime = "text/plain";
att.file.path = "/text_file.txt";
message.addAttachment(att);

/* Connect SMTP */
if (!smtp.connect(&config)) {
    Serial.println("SMTP connection failed");
    Serial.println(smtp.errorReason());
    return;
}

/* Send mail */
if (!MailClient.sendMail(&smtp, &message)) {
    Serial.println("Email send failed");
    Serial.println(smtp.errorReason());
}
}

void loop() {
    // nothing
}

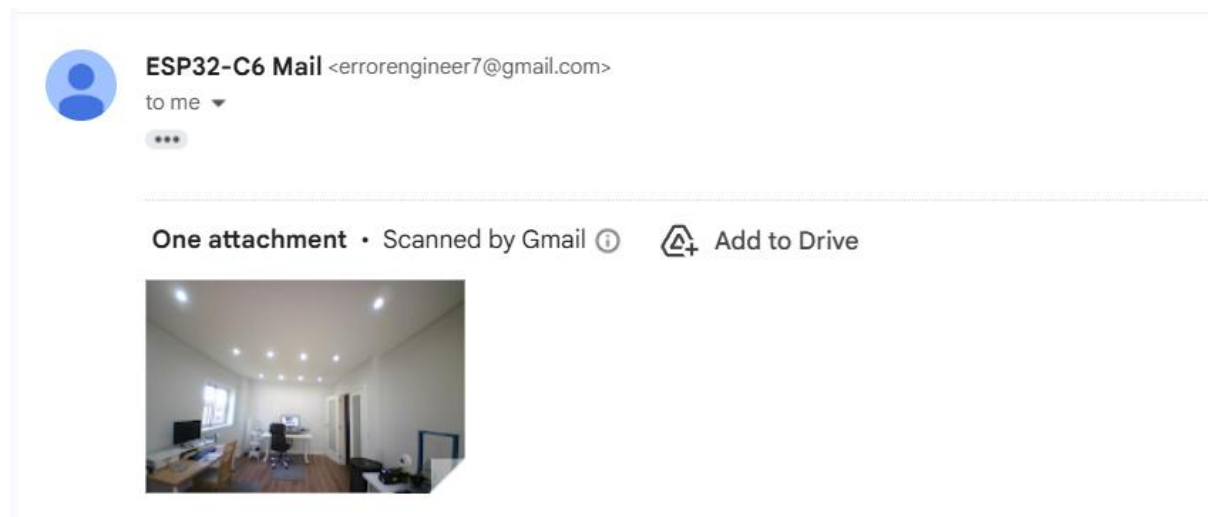
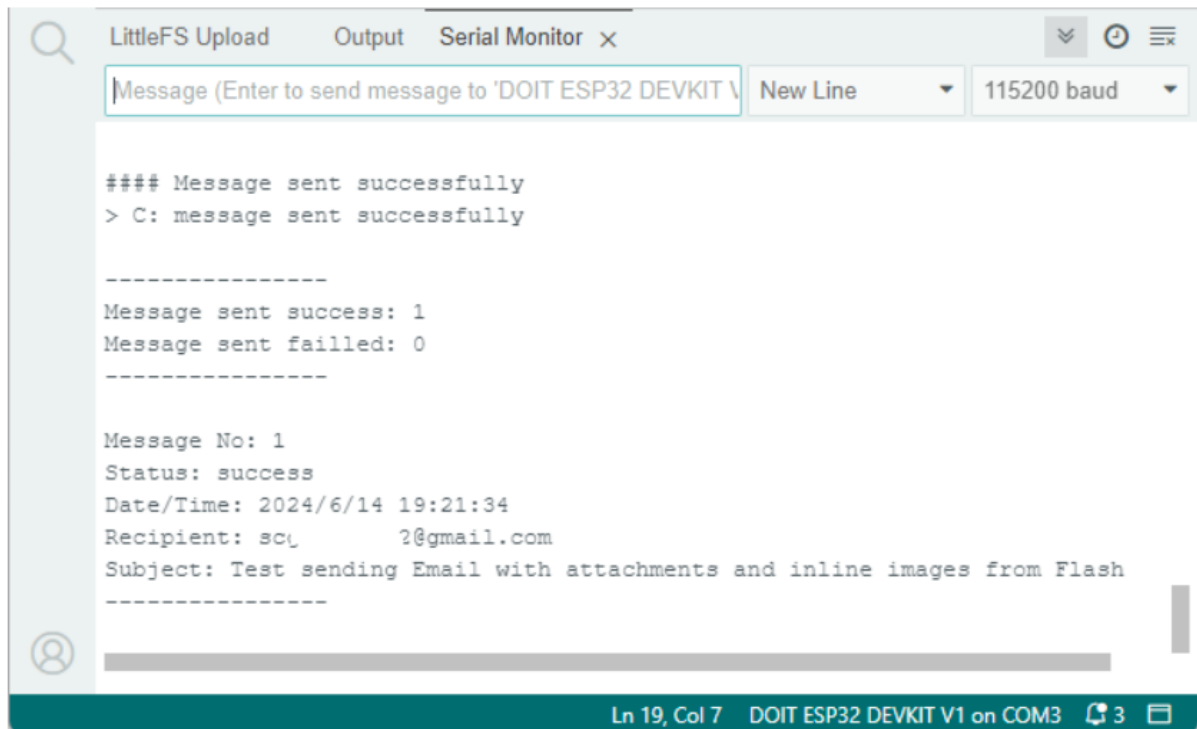
/* Callback */
void smtpCallback(SMTP_Status status) {
    Serial.println(status.info());

    if (status.success()) {
        Serial.println("Email sent successfully ");
        smtp.sendingResult.clear();
    }
}

```


Result:

- After uploading the code, open the Serial Monitor at a baud rate of 115200 and press the on-board RESET button. If everything goes smoothly, you should get a similar message on the Serial Monitor.



Tasks:

- Send a Email with Attachment (jpg or txt) to multiple recipient emails and check whether recipient received or not.